Relatorio de Tecnologias
No Soberano - PIDE / FAPESPA

o papel de cada tecnologia na arquitetura
e a resposta em uma frase para cada 'o que isso faz?'

Stack real da malha de dois nos (deploy/malha/) ?
a mesma imagem em cada instituicao; muda o papel,
nao a stack.

1. Visao geral da stack

Cada no soberano e a MESMA composicao de servicos. O que muda entre instituicoes e o papel na tramitacao (origem ou destino), nunca a stack. Esta e a arquitetura-alvo: cada orgao opera esta composicao na propria infraestrutura.

Camada	Tecnologia	Funcao no no
API	FastAPI (Python)	tramitacoes, upload, recebimentos, acks, verificacao
Banco	PostgreSQL	metadados, trilha hash-chain, outbox, replay_nonces ? SEM bytes
Fila	RabbitMQ (TLS)	esteira assincrona de anonimizacao ? quorum + DLQ
Storage	MinIO (HTTPS)	objetos ? sempre ciphertext E2E em repouso
Borda	nginx + mTLS	identidade institucional por certificado de cliente
Segredos	Vault	custodia da chave E2E ? AppRole, rotacao, revogacao
Worker	Python + MotorIA	anonimizacao via porta permutavel
Relay	Python	outbox -> broker
Cripto	ChaCha20-Poly1305, Ed25519, SHA-256	E2E nos 3 pontos de I/O; assinaturas; integridade
Plataforma	Docker Compose, Alembic, pytest	malha de 2 nos; migracoes; suite smoke/e2e

Regra que atravessa tudo: o binario do documento NUNCA entra em banco, envelope, broker ou resposta de API ? so metadados + SHA-256 circulam entre os nos.

2. Os servicos do no ? um a um

FastAPI (a API)

Papel na arquitetura: a superficie publica do no ? todas as operacoes de tramitacao passam por ela.

O que e: framework web Python assincrono. Recebe criacao de tramitacao, upload multipart, recebimento de envelope, acks e o endpoint de verificacao da cadeia.

Por que ela: mesma linguagem do worker e do motor ? uma base so de codigo; async aguenta I/O concorrente sem complexidade.

Onde ver: app/main.py, app/service.py, app/deps.py.

> *>> e a porta do no: tudo que a instituicao ou outra instituicao pede ao sistema entra pela API ? e ela aplica as regras antes de tocar storage ou banco.*

PostgreSQL (o banco)

Papel na arquitetura: guarda so metadados e evidencias ? nunca o documento.

O que e: registros de tramitacao, trilha de auditoria (hash-chain), outbox de eventos, replay_nonces e o cofre de pseudonimos.

Por que ela: transacoes: o upload grava objeto + trilha + outbox numa UNICA transacao ? nao existe 'arquivou mas nao registrou'. E o trigger append-only barra UPDATE/DELETE na trilha no nivel do banco.

Onde ver: alembic/versions/ (migracoes ? 0007 = append-only); app/repository.py.

> *>> o banco e o livro-razao: metadados, hashes e a corrente de auditoria ? e por desenho nao existe sequer coluna para guardar bytes de documento.*

RabbitMQ (a fila)

Papel na arquitetura: a esteira assincrona ? desacopla a API do processamento pesado de anonimizacao.

O que e: broker de mensagens so em TLS (porta 5671). Fila quorum com DLQ: evento de 'documento arquivado' dispara o worker sem a chamada de upload ficar esperando.

Por que ela: anonimizacao pode demorar ? a API responde na hora e o processamento continua em segundo plano. Fila quorum sobrevive a restart; DLQ guarda o que falhar de vez.

Onde ver: deploy/malha/rabbitmq.conf (TLS-only); app/worker.py (consumidor); app/relay.py (produtor via outbox).

> *>> e a correia interna do no: o evento sai da API, viaja cifrado pela fila e acorda o worker ? tudo dentro do dominio da instituicao.*

MinIO (o storage)

Papel na arquitetura: o cofre de objetos da instituicao ? guarda o documento e depois o anonimizado.

O que e: object storage compativel com S3, servido em HTTPS. Dois buckets por no: <org>-docs e <org>-docs-anonimizados.

Por que ela: soberania fisica: cada instituicao e dona do proprio storage. E em repouso so existe ciphertext ? o objeto cru comeca com a magia E2 (0xE2 0x02), legivel so dentro do processo.

Onde ver: app/storage.py (ArmazenamentoCifrado); prova no terminal: 'objeto cru no MinIO' na demo.

> *>> e o cofre da instituicao: quem olha o MinIO por dentro ve so ciphertext ? o documento em claro nunca repousa em lugar nenhum.*

nginx + mTLS (a borda)

Papel na arquitetura: a fronteira institucional ? quem bate na porta precisa de certificado de cliente valido.

O que e: proxy reverso que TERMINA o mTLS: verifica o certificado da instituicao cliente e injeta X-Cliente-Orgao. Aplica rate limit e HSTS.

Por que ela: identidade institucional por criptografia de certificado ? substituiu JWT (reprovado: token em browser e vulneravel a XSS). mTLS identifica o ORGAO, nao um usuario.

Onde ver: deploy/malha: nginx.conf + certs gerados por scripts/gerar_certs_mtls.py.

> *>> e a portaria do predio: so entra quem apresenta credencial de instituicao reconhecida ? e a borda ja carimba de que orgao e a requisicao.*

Vault (os segredos)

Papel na arquitetura: a custodia das chaves ? a chave E2E mora la, nao no codigo nem em arquivo.

O que e: servidor de segredos (KV v2): a API e o worker leem a chave E2E via AppRole ? login com role_id + secret_id, token curto de 60s renovavel. Suporta rotacao (atual + anterior_N) e revogacao.

Por que ela: separar a chave do codigo: da para girar a chave sem trocar deploy, revogar sem recompilar, e auditar quem leu o que. E2E_FONTE=env|vault ? env so para dev.

Onde ver: app/cofre.py; deploy/malha/vault-init.sh (policy e2e-leitura + role no-a).

> *>> e o cofre de chaves do predio: a chave que decifra os envelopes nao fica escrita em lugar nenhum do codigo ? cada servico pede emprestada ao Vault e ela tem prazo de validade.*

Worker + porta MotorIA

Papel na arquitetura: o anonimizador ? roda DENTRO do no, a documento nunca sai para ser tratado.

O que e: processo que consome a fila, baixa a copia cifrada, aplica o motor e publica no bucket anonimizados ? depois do gate de PII residual (fail-closed: sobrou PII, nao grava).

Por que ela: MotorIA e uma PORTA permutavel: anonimizar(bytes)->bytes. Trocar o motor (regex, modelo medido, NER discriminativo futuro) nao toca worker, fila ou buckets.

Onde ver: app/worker.py; implementacoes: DemoMotorIA (demo), app/motor_ollama.py (POC medida).

> *>> e o funcionario que trata o documento no arquivo do proprio orgao: o motor e peca trocavel ? a esteira ao redor nao muda.*

Relay (o outbox)

Papel na arquitetura: a ponte transacional entre banco e fila.

O que e: varre a tabela outbox e publica no RabbitMQ ? o evento e gravado junto com a operacao na mesma transacao e o relay garante a entrega ao broker (com retry).

Por que ela: resolve o problema classico: sem outbox, 'commitou no banco mas caiu antes de avisar a fila' perderia o evento. Com outbox, gravar e anunciar sao atomicos.

Onde ver: app/relay.py; migracao 0005 (outbox retry).

> *>> e o carteiro interno: o que o banco registra, o relay garante que chega a fila ? nenhum evento se perde entre a gravacao e o processamento.*

3. As primitivas de seguranca

Nao sao 'servicos' mas sao o que a plateia pergunta quando ve 'E2E', 'assinatura' e 'hash' na tela.

ChaCha20-Poly1305 (o E2E)

Papel na arquitetura: a blindagem ponta a ponta ? tudo que atravessa ou repousa vai dentro de quadros AEAD.

O que e: cifra autenticada: cada quadro e 0xE2|0x02|nonce|len|ts_ms|ct ? o timestamp entra no AAD (adultera-lo quebra o selo). Cobre os 3 pontos de I/O: request/response HTTP, mensagens do broker e objetos no storage.

Por que ela: AEAD = confidencialidade + integridade no mesmo selo; ChaCha20 e rapido em software puro (sem AES-NI). O formato versionado permite evoluir o quadro.

Onde ver: app/crypto_e2e.py (selar/abrir), app/middleware_e2e.py (HTTP), PublicadorRabbitMQ (broker), ArmazenamentoCifrado (storage).

> *>> e o envelope lacrado de ponta a ponta: mesmo que alguem capture o pacote no fio ou abra o MinIO, ve so ciphertext ? e qualquer alteracao estoura o lacre.*

Ed25519 (as assinaturas)

Papel na arquitetura: a prova de autoria ? envelopes e acks sao assinados pela chave da instituicao.

O que e: assinatura digital de curva eliptica: A assina TODOS os campos do envelope; B verifica contra a chave publica de A antes de persistir. O ack e assinado por B e verificado por A.

Por que ela: deterministica, rapida, chaves curtas ? e fail-closed por construcao: uma virgula adulterada invalida tudo.

Onde ver: app/assinatura.py; chaves ORG_SIGN_*/ORGAO_PUBKEYS_* no .env da malha (gitignored).

> *>> e o selo de cera da instituicao: quem recebe sabe DE QUEM veio e que ninguem mexeu ? por isso o envelope adulterado da demo e jogado fora.*

SHA-256 (integridade + cadeia)

Papel na arquitetura: a impressao digital do documento e o elo da corrente de auditoria.

O que e: hash criptografico: declarado no criar, conferido no upload, revalidado no download e recomputado por B no proprio storage antes de assinar. Na trilha, cada elo e sha256(elo anterior | evento).

Por que ela: deterministico e publico ? qualquer ponta recomputa e confere sem confiar na outra. Encadear transforma uma lista de eventos em uma corrente: mexer num elo quebra todos os seguintes.

Onde ver: app/repository.py (encadear_uma_vez); GET /verificar rejoga do GENESIS.

> *>> e a impressao digital: nao prova que voce tem o documento certo porque alguem disse ? prova porque os 64 hexadecimais batem, e cada um pode conferir sozinho.*

replay_nonces (anti-replay)

Papel na arquitetura: o carimbo unico ? um pacote cifrado so vale uma vez.

O que e: tabela no Postgres onde cada nonce de quadro E2E e reivindicado atomicamente (PK) antes de processar; janela de frescura E2E_JANELA_MS = 300s.

Por que ela: capturar e reentregar o MESMO pacote nao pode repetir a operacao ? o claim atomico garante isso ate com replicas. AMQP so usa frescura (redelivery legitima repete o envelope).

Onde ver: middleware_e2e (claim); migracao de replay_nonces; passo 'replay' da demo prova 200 -> 400.

> *>> e o carimbo de correio: o pacote chega, recebe o carimbo unico ? a fotocopia dele depois e rejeitada porque esse carimbo ja foi gasto.*

4. Plataforma e motor ? o que sustenta

Docker Compose (a malha)

Papel na arquitetura: a bancada que prova tudo: 2 nos reais, ~16 conteineres.

O que e: cada no = api + postgres + rabbitmq + minio + worker + relay + nginx; mais vault e o conteiner peer que dirige a demo. Redes separadas: borda (mTLS) e interna.

Por que ela: reproduzivel: docker compose up e a arquitetura inteira existe ? com TLS real em todos os links internos (postgres ssl, rabbitmq 5671, minio https, vault https).

Onde ver: deploy/malha/compose.malha.yaml (+ compose.demo.yaml para o motor da demo).

> *>> e a maquete que funciona de verdade: nao e desenho da arquitetura, e a arquitetura executando ? cada orgao real rodaria essa mesma composicao na sua infra.*

Alembic (migracoes)

Papel na arquitetura: a evolucao versionada do banco.

O que e: migracoes incrementais: 0005 = outbox retry, 0007 = trigger append-only + replay_nonces.

Por que ela: o schema nao e arte manual: sobe junto com a aplicacao (alembic upgrade head no boot da api).

Onde ver: alembic/versions/.

> *>> e o diario de obras do banco: cada mudanca de estrutura e uma migracao versionada e aplicada na subida.*

Motor de anonimizacao ? a ressalva honesta

Papel na arquitetura: a peca que remove PII ? trocavel por desenho.

O que e: na DEMO roda DemoMotorIA (regex LGPD deterministica) ? a anonimizacao fica VISIVEL e repetivel ao vivo. O candidato medido e a cascata regex+Ollama qwen2.5: NER exact F1 0,953, reidentificacao 0/360 ? no corpus SINTETICO.

Por que ela: separar 'provar a esteira' de 'medir o modelo': a demo precisa de determinismo; a qualidade e medida a parte em docs/metricas.md. A cascata e desvio documentado da ADR-0011 (zero IA generativa) ? producao exige NER discriminativo treinado no mesmo gate.

Onde ver: app/motor_ollama.py; docs/metricas.md; ADR-0011.

> *>> na demonstracao o motor e o didatico ? prova que a esteira inteira funciona. A nota do modelo real esta medida no relatorio de metricas e ainda nao esta plugada na porta.*

5. Cola de perguntas ? 'o que o X faz?'

Uma linha por tecnologia, na ordem em que costumam perguntar:

Pergunta	Resposta em uma frase
o que o Vault faz?	guarda a chave E2E fora do codigo ? os servicos pedem emprestada com token de 60s; da pra girar e revogar sem deploy
o que o RabbitMQ faz?	a fila interna do no ? desacopla a API da anonimizacao pesada; quorum + DLQ
o que o MinIO faz?	o cofre de objetos da instituicao ? guarda o documento e o anonimizado, sempre so ciphertext
o que o nginx faz?	a borda mTLS ? exige certificado de instituicao e carimba de que orgao e a requisicao
o que o Postgres faz?	o livro-razao ? metadados, trilha hash-chain e nonces; sem coluna para bytes de documento
o que o FastAPI faz?	a porta do no ? recebe tudo e aplica as regras antes de tocar banco ou storage
o que o worker faz?	o anonimizador interno ? consome a fila e publica o objeto tratado, com gate de PII residual
o que o relay faz?	o carteiro outbox ? o que o banco grava, ele garante que chega a fila
o que e o 'E2E'?	quadros ChaCha20-Poly1305 em HTTP, fila e storage ? lacrado + cifrado de ponta a ponta
o que e o Ed25519?	a assinatura da instituicao em envelopes e acks ? uma virgula adulterada invalida tudo
o que e a hash-chain?	cada evento vira sha256(elo anterior | evento) ? mexer num elo quebra todos os seguintes
o que sao os nonces?	o carimbo unico por pacote ? a fotocopia reenviada e recusada porque o carimbo ja foi gasto